AyED II

Tablas de Dispersión (Hash)

Ingeniería en Sistemas de Información
2° Año -1° Cuatrimestre

Tablas de Dispersión

Que es una Tabla de dispersión?

- Es una estructura de datos que almacena elementos usando una clave.
- Cada elemento tiene una clave única (no se repite).
- La clave suele ser un número entero dentro de un rango acotado.
- Permite organizar datos de forma eficiente para acceder rápidamente.

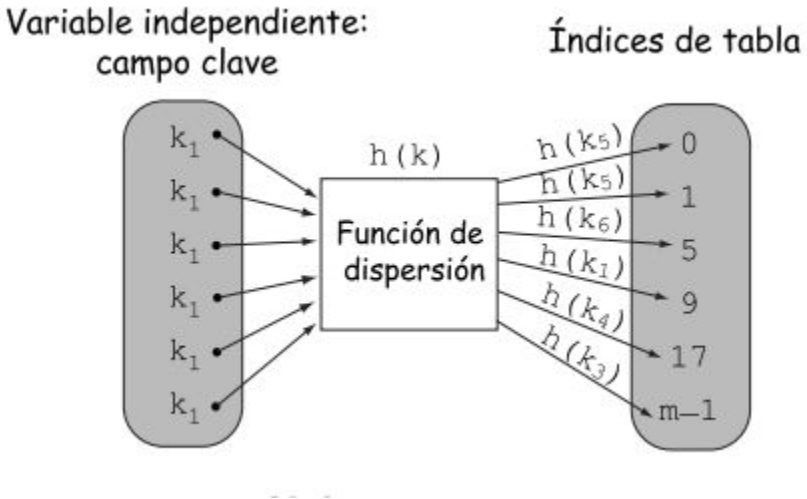
Tablas de Dispersión

Un Diccionario (TAD) es una estructura que:

- Asocia una clave única a cada elemento.
- No permite claves repetidas.

Las tablas de dispersión son una forma eficiente de implementar un diccionario.

Tablas de Dispersión



Tablas de Dispersión

Una tabla de dispersión consta de un array (vector), donde se almacenan los registros o elementos, y de una función hash, que transforma el campo clave elegido en el rango entero del array.

Operaciones en Tablas de Dispersión

Las tablas de dispersión implementan eficientemente el TAD Diccionario.

Un diccionario:

- Asocia una clave con un valor
- Cada clave permite acceder a su valor

Diccionario de inglés:

- clave -> palabra
- valor -> traducción

Operaciones en Tablas de Dispersión

En una tabla de dispersión:

Insertar : Agrega un elemento según su clave

Buscar : Encuentra un elemento usando su clave

Eliminar : Quita un elemento de la tabla

La clave NO se usa directamente
Se aplica una función:

$h(x) \rightarrow$ índice entonces tabla[h(x)]

Eficiencia y Problemas

Ventaja principal

Operaciones rápidas -> **O(1)** (tiempo constante)

Problema **Colisiones**

Dos claves distintas pueden ir al mismo índice: $h(x_i) = h(x_j)$

Se necesita un método para resolverlas

Ejemplo

Datos:

Tabla: tamaño = 10 -> índices 0 a 9

Función hash: Suma de letras (A=1…Z=26) % 10

Consigna

1. Calcular el valor numérico de cada letra
2. Sumar los valores del nombre
3. Aplicar módulo 10
4. Insertar el nombre en la posición correspondiente

Ejemplo

Ana
A=1, N=14, A=1
Suma = 16 → 16 % 10 = 6
Insertar en tabla[6]

Luis
L=12, U=21, I=9, S=19
Suma = 61 → 61 % 10 = 1
Insertar en tabla[1]

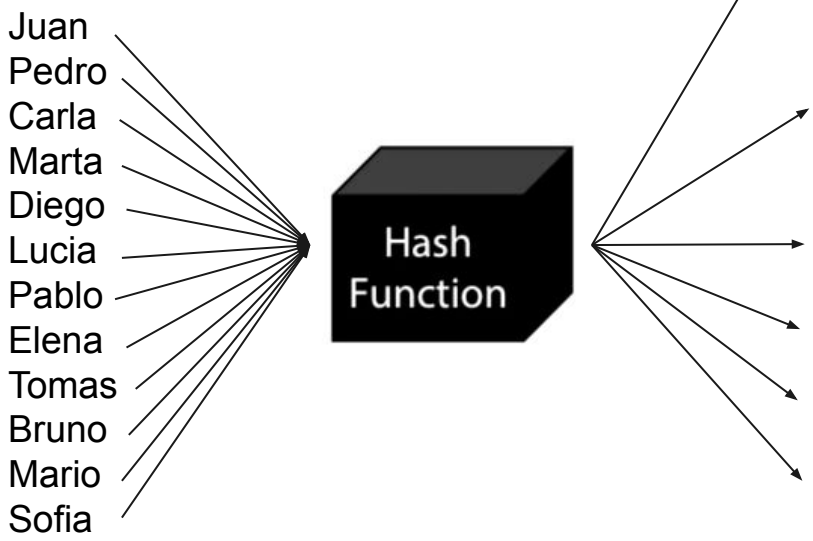
Sofia
S=19, O=15, F=6, I=9, A=1
Suma = 50 → 50 % 10 = 0
Insertar en tabla[0]

TABLA

0	Sofia
1	Luis
2	
3	
4	
5	
6	Ana
7	
8	
9	

Juan
Pedro
Carla
Marta
Diego
Lucia
Pablo
Elena
Tomas
Bruno
Mario
Sofia

Ejemplo



0	Sofia → Diego → Bruno
1	
2	
3	Marta
4	
5	Carla
6	Juan → Lucia → Pablo → Mario
7	Elena
8	Pedro → Tomas
9	

Eficiencia y Problemas

Factor de carga

$\lambda = n / m$

n → cantidad de elementos

m → tamaño de la tabla

Recomendación:

$\lambda \leq 0.8$

Menos colisiones (Bueno)
Más memoria libre (Malo)

Funciones Hash

$$h: U \rightarrow \{0,1,2,\ldots,m-1\}$$

h es una función que va desde el conjunto de claves U al conjunto de índices de la tabla

h → función hash
U → universo de claves posibles
{0,1,...,m−1} → posiciones de la tabla
m → tamaño de la tabla

Propiedades de las Funciones Hash

- **Determinista** : Una función hash debe producir consistentemente la misma salida para la misma entrada..
- **Tamaño de salida fijo** : La salida de una función hash debe tener un tamaño fijo, independientemente del tamaño de la entrada.
- **Eficiencia** : La función hash debe ser capaz de procesar la entrada rápidamente.
- **Uniformidad** : La función hash debe distribuir los valores hash de manera uniforme en todo el espacio de salida para evitar la agrupación.
- **Resistencia a la preimagen** : Debe ser computacionalmente inviable invertir la función hash, es decir, encontrar la entrada original dado un valor hash.
- **Resistencia a colisiones** : Debería ser difícil encontrar dos entradas diferentes que produzcan el mismo valor hash.
- **Efecto avalancha** : Un pequeño cambio en la entrada debería producir un valor hash significativamente diferente

Aplicaciones de las funciones hash

- **Tablas hash** : El uso más común de las funciones hash en DSA es en las tablas hash, que proporcionan una forma eficiente de almacenar y recuperar datos.
- **Integridad de los datos** : Las funciones hash se utilizan para garantizar la integridad de los datos mediante la generación de sumas de verificación.
- **Criptografía** : En las aplicaciones criptográficas, las funciones hash se utilizan para crear algoritmos hash seguros como SHA-256.
- **Estructuras de datos** : Las funciones hash se utilizan en diversas estructuras de datos, como los filtros de Bloom y los conjuntos hash.

Aritmética modular / Método de división

Una función de dispersión que utiliza la aritmética modular genera valores dispersos calculando el resto de la división entera entre la clave x y el tamaño de la tabla m.

$$h(x) = x \text{ modulo } m$$

La operación resto(módulo) genera un numero entre 0 y m-1 cuando el segundo operando es m.

Por tanto, esta función de dispersión proporciona valores enteros dentro del rango 0 ... m-1.

Aritmética modular / Método de división

Con el fin de que esta función disperse lo más uniformemente posible, es necesario tener ciertas precauciones con la elección del tamaño de la tabla, m.

- No es recomendable escoger el valor de m múltiplo de 2 ni tampoco de 10.
- Las elecciones recomendadas del tamaño de la tabla, m, son números primos mayores, aunque cercano al número de elementos, n , que tiene previsto que almacene la tabla.

Ventajas :

- Fácil de implementar.
- Funciona bien cuando *m* es un número primo.

Desventajas :

- Distribución deficiente si *m* no se elige sabiamente.

Ejemplo

mala elección de m

Supongamos:

- m=10 (NO primo)
- Claves: **20, 30, 40, 50, 60**

buena elección de m

Ahora cambiamos:

- m=11 (primo)
- mismas claves: **20, 30, 40, 50, 60**

Cálculos

- 20 mod 10=0
- 30 mod 10=0
- 40 mod 10=0
- 50 mod 10=0
- 60 mod 10=0

Cálculos

- 20 mod 11=9
- 30 mod 11=8
- 40 mod 11=7
- 50 mod 11=6
- 60 mod 11=5

Plegamiento

Consiste en partir la clave **x** en varias partes: **x**1, **x2**, **x**3 ... **x**n.

La función hash se define de la siguiente forma:

$$h(x) = x1 + x2 + x3 + ... + xr$$

El método de plegado se utiliza para convertir una clave muy extensa en una más manejable, dividiéndola y combinándola, y posteriormente aplicar una función hash mediante aritmética modular.

La operación que se realiza para el cálculo de la función hash desprecia los dígitos mas significativos obtenidos del acarreo.

Plegamiento

A veces se determina la inversa de las partes pares, **x**2, **x**4 ... antes de sumarlas, con el fin de conseguir mayor dispersión. La inversa en el sentido de invertir el orden de los dígitos.

Pasos :

1. Divide la llave en partes.
2. Suma las partes.
3. Toma el módulo *m* de la suma.

Ventajas :

- Sencillo y fácil de implementar.

Desventajas :

- Depende del esquema de particionamiento elegido.

Ejemplo

Clave: 123456

- 123 | 456
- invertimos segunda → 654
- suma: 123 + 654 = **777**
- 777 mod 10=7

Clave: 345612

- 345 | 612 → 216
- suma: 345 + 216 = **561**
- 561 mod 10=1

Clave: 561234

- 561 | 234 → 432
- suma: 561 + 432 = **993**
- 993 mod 10=3

Mitad del cuadrado

Esta técnica de dispersión consiste en calcular el cuadrado de la clave **x** y, a partir del resultado **x**^2, seleccionar determinados dígitos —generalmente los centrales— para obtener la dirección hash.

El número de dígitos a extraer depende del rango de dispersión que se quiera obtener.

Pasos :

1. Calcular el cuadrado de la llave.
2. Extrae los dígitos centrales del valor al cuadrado. *(siempre los mismos)*

Ventajas :

- Produce una buena distribución de valores hash.

Desventajas :

- Puede requerir un mayor esfuerzo computacional.

Ejemplo

calcular cuadrados

- 1234^2 = 1522756
- 1235^2 = 1525225
- 1236^2 = 1527696

tomar 3 dígitos centrales

- 1522756 → **227**
- 1525225 → **252**
- 1527696 → **276**

tomar últimos 3 dígitos

- 1522756 → **756**
- 1525225 → **225**
- 1527696 → **696**

Problema real

Si probamos otras similares:

- $2234^2 = 4990756$ → **756**
- $3234^2 = 10458756$ → **756**

Método de la multiplicación

El método de la multiplicación para obtener direcciones hash se desarrolla en tres pasos. Primero, se multiplica la clave **x** por una constante real **R**, donde $0<\mathbf{R}<1$. Luego, se toma la parte decimal **d** del resultado de esa multiplicación ($\mathbf{R} \cdot \mathbf{x}$). Finalmente, se multiplica **d** por el tamaño de la tabla **m** y se trunca el resultado para obtener un número entero en el rango **0** a **m−1**, que corresponde a la dirección hash.

Una elección de la constante **R** es la inversa de la razón áurea, **R** = 0.6180334.

Este método de dispersión presenta dos características importantes. En primer lugar, dos claves que contienen los mismos dígitos en distinto orden no tienen mayor probabilidad de producir una colisión que cualquier otro par de claves. En segundo lugar, dos claves con valores numéricos muy cercanos pueden generar direcciones hash muy alejadas entre sí dentro de la tabla.

Método de la multiplicación

Pasos:

1. R * x
2. d = R * x – ParteEntera(R * x)
3. h(x) = ParteEntera(m * d)

Ventajas :

- Menos sensible a la elección de *m* .

Desventajas :

- Más complejo que el método de división.

Ejemplo

R = 0.6180334
m = 1000 (queremos 3 dígitos)

Claves:

- 1234
- 4321

Paso 1: multiplicar por R

- 1234 x 0.6180334 ≈ 762.65
- 4321 x 0.6180334 ≈ 2670.91

Paso 2: tomar parte decimal

- 762.65 → **0.65**
- 2670.91 → **0.91**

Paso 3: multiplicar por m = 1000

- 0.65 x 1000 = 650
- 0.91 x 1000 = 910

Funciones hash criptográficas

Las funciones hash criptográficas están diseñadas para ser seguras y se utilizan en criptografía. Algunos ejemplos son MD5, SHA-1 y SHA-256.

Características :

- Resistencia a la preimagen.
- Segunda resistencia de preimagen.
- Resistencia a las colisiones.

Ventajas :

- Alta seguridad.

Desventajas :

- Requiere mucha capacidad de cálculo.

Hashing universal

El hash universal utiliza una familia de funciones hash para minimizar la probabilidad de colisión para cualquier conjunto de entradas dado.

$$h(k) = ((a \cdot k + b) \bmod p) \bmod m$$

Donde a y b son constantes elegidas al azar, p es un número primo mayor que m y k es la clave.

Ventajas :

- Reduce la probabilidad de colisiones.

Desventajas :

- Requiere mayor capacidad de procesamiento y almacenamiento.

Hashing perfecto

El hash perfecto busca crear una función hash libre de colisiones para un conjunto estático de claves. Garantiza que no habrá dos claves que produzcan el mismo valor hash.

Tipos :

- Hash perfecto mínimo: garantiza que el rango de la función hash sea igual al número de claves.
- Hashing perfecto no mínimo: el rango puede ser mayor que el número de claves.

Ventajas :

- Sin colisiones.

Desventajas :

- Complejo de construir.

Colisiones y Resolución de Colisiones

La operación de inserción debe contemplar mecanismos para resolver colisiones de manera eficiente.

De igual manera las operaciones de busqueda y eliminacion tambien quedan afectadas por la resolucion de colisiones

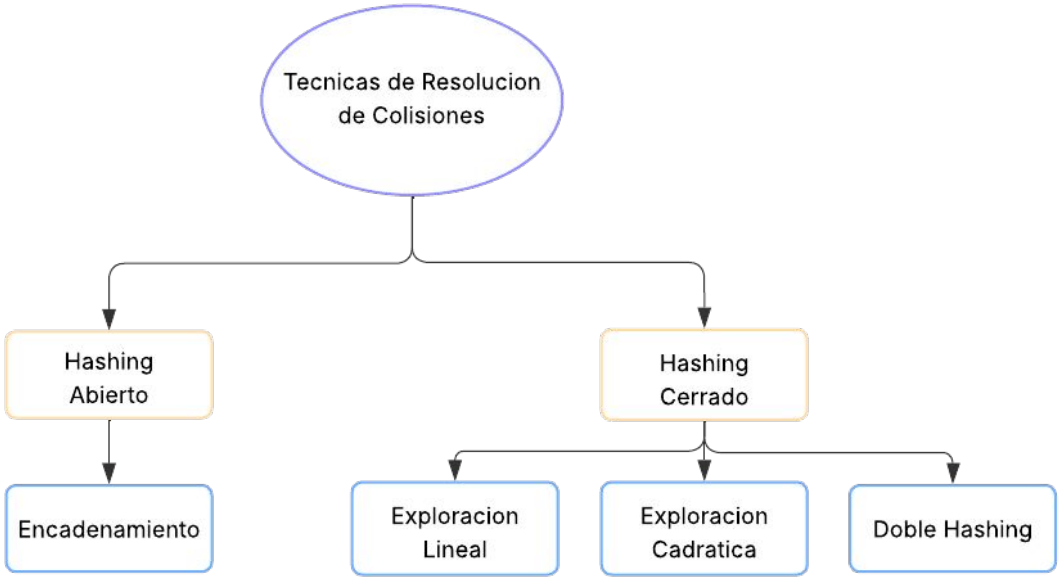
Colisiones y Resolución de Colisiones

Hay dos cuestiones que se deben considerar a la hora de diseñar una tabla hash. Primero, seleccionar una buena función hash, que disperse lo mas uniformemente. Segundo, seleccionar un método para resolver colisiones.

Empleamos estrategias de resolución de colisiones para lidiar con esta colisión. Generalmente existen dos tipos de técnicas de resolución de colisiones:

1. Hashing abierto.
2. Hashing cerrado. (Direccionamiento abierto)

Colisiones y Resolución de Colisiones



Exploracion de Direcciones

La exploración de direcciones es una técnica de resolución de colisiones utilizada en tablas hash con direccionamiento abierto. En este enfoque, todos los elementos se almacenan dentro de la misma tabla.

Cuando ocurre una colisión, en lugar de usar estructuras externas, se busca otra posición disponible siguiendo una secuencia de direcciones definida por un método de exploración.

Este proceso se aplica tanto en inserción como en búsqueda y eliminación, recorriendo la tabla hasta encontrar una posición adecuada.

Inserción y Secuencia de Exploración

Al insertar un elemento, primero se calcula su posición inicial mediante la función hash $i=h(x)$.

Si la posición está ocupada, se produce una colisión y se inicia la exploración de direcciones, recorriendo posiciones según una secuencia (por ejemplo, lineal o cuadrática) hasta encontrar un espacio libre.

La inserción finaliza cuando se encuentra una posición vacía, previamente inicializada con un valor especial como null, que indica que nunca fue utilizada.

Búsqueda de Elementos

Para buscar un elemento, se calcula su posición inicial mediante la función hash.

Si el elemento no se encuentra en esa posición, se continúa explorando la secuencia de direcciones utilizada en la inserción, comparando los elementos almacenados.

La búsqueda finaliza cuando:

- se encuentra el elemento buscado, o
- se llega a una posición vacía (null), lo que indica que el elemento no está en la tabla.

Este comportamiento depende de que las posiciones vacías estén correctamente inicializadas.

Eliminación y Marcas de Borrado

En la operación de eliminación, una vez encontrado el elemento, no se debe dejar la posición vacía directamente, ya que esto interrumpiría la secuencia de exploración.

En su lugar, se utiliza una marca especial (por ejemplo, “borrado” o *tombstone*) para indicar que la posición fue ocupada anteriormente.

Esta marca permite que futuras búsquedas continúen correctamente la exploración, evitando errores en la localización de elementos.

Además, las posiciones marcadas como borradas pueden reutilizarse en operaciones de inserción posteriores.

Exploración Lineal

La exploración lineal es el método más simple para resolver colisiones en tablas hash con direccionamiento abierto, donde todos los elementos se almacenan en la misma tabla.

Cuando dos claves generan la misma posición mediante la función hash, se produce una colisión. Para resolverla, se exploran posiciones consecutivas hasta encontrar una libre o el elemento buscado, recorriendo la tabla de forma secuencial.

Este enfoque es sencillo de implementar, pero puede generar agrupamiento de elementos, afectando el rendimiento.

Fórmula general de exploración lineal:

$$H(clave,i)=(h(clave)+i) \bmod m$$

Secuencia de Exploración

Dada una clave x, se calcula la posición inicial:

$$p=h(x)$$

Si la posición está ocupada, se inicia la exploración lineal evaluando sucesivamente:

$$(p+i) \bmod m \qquad \text{con } i=0,1,2,\ldots$$

De esta forma, se recorren posiciones consecutivas de la tabla. El uso del módulo garantiza la circularidad, permitiendo continuar desde el inicio al alcanzar el final.

La inserción finaliza al encontrar la primera posición libre, mientras que la búsqueda continúa hasta hallar el elemento o una posición vacía.

Búsqueda y Eliminación

Las operaciones de búsqueda y eliminación siguen la misma secuencia que la inserción.

La búsqueda comienza en la posición h(x). Si el elemento no se encuentra, se continúa explorando secuencialmente hasta encontrarlo o hasta llegar a una posición vacía, lo que indica que el elemento no está en la tabla.

La eliminación requiere primero localizar el elemento mediante la búsqueda y luego marcar la posición como borrada (no vacía), para no interrumpir la secuencia de exploración.

Problema del Agrupamiento (Clustering)

La exploración lineal presenta un inconveniente importante: el agrupamiento de elementos en posiciones contiguas.

Cuando varias colisiones ocurren en zonas cercanas, se forman bloques de elementos consecutivos. Esto provoca que nuevas inserciones tiendan a agrandar esos bloques, aumentando la cantidad de posiciones que deben explorarse.

Este fenómeno se intensifica a medida que el factor de carga aumenta, afectando negativamente el rendimiento.

Factor de Carga y Rendimiento

El factor de carga λ= n/m es determinante en la eficiencia de la exploración lineal.

Cuando λ es bajo, hay muchas posiciones libres y las operaciones son rápidas. Sin embargo, cuando λ se aproxima a 0.5 o lo supera, el agrupamiento de elementos se vuelve significativo.

Esto provoca que el número de posiciones examinadas en inserciones y búsquedas aumente considerablemente.

Análisis de Complejidad

La eficiencia de las operaciones se mide por la cantidad de posiciones examinadas.

Para la búsqueda se consideran dos casos:

- S(λ): número medio de comparaciones en búsquedas con éxito
- U(λ): número medio de comparaciones en búsquedas sin éxito

En exploración lineal, ambas crecen rápidamente a medida que aumenta λ, especialmente cuando se generan agrupamientos.

Esto implica que el rendimiento puede degradarse significativamente si no se controla el factor de carga.

Conclusión y Consideraciones

La exploración lineal es un método simple y fácil de implementar, pero su principal desventaja es el agrupamiento de elementos.

Cuando el factor de carga supera aproximadamente el 50%, el rendimiento de las búsquedas disminuye debido al aumento en la longitud de las secuencias exploradas.

Por esta razón, es fundamental mantener controlado el factor de carga y utilizar técnicas como el rehashing para preservar la eficiencia de la tabla hash.

Exploración Cuadrática

La exploración cuadrática es un método de resolución de colisiones en tablas hash con direccionamiento abierto que busca evitar el problema de agrupamiento (clustering) propio de la exploración lineal.

Cuando ocurre una colisión, en lugar de recorrer posiciones consecutivas, se utilizan saltos crecientes basados en cuadrados, lo que permite dispersar mejor los elementos en la tabla.

$$H(clave,i) = (h(x)+c1\ i+c2\ i2)\ \ mod\ \ \ m$$

Secuencia de Exploración

Dada una clave x, se calcula:

$$p=h(x)$$

Si la posición está ocupada, se utiliza la siguiente secuencia:

$$p,\ p+1^2,\ p+2^2,\ p+3^2,\ldots$$

Es decir:

$$p,\ p+1,\ p+4,\ p+9,\ldots$$

La tabla se considera circular, por lo que las posiciones se calculan módulo m.

Ventajas y Limitaciones de la Exploración Cuadrática

La exploración cuadrática presenta como principal ventaja la reducción del **agrupamiento primario** (*primary clustering*) característico de la exploración lineal.

La exploración cuadrática utiliza saltos crecientes, lo que dispersa los elementos y evita la formación de estos agrupamientos consecutivos.

Sin embargo, este método introduce una limitación importante: la secuencia de exploración no necesariamente recorre todas las posiciones de la tabla. Esto implica que pueden existir posiciones libres que nunca sean alcanzadas por la secuencia, impidiendo la inserción de nuevos elementos a pesar de haber espacio disponible.

Este comportamiento depende tanto del tamaño de la tabla como del factor de carga.

Condiciones para un Correcto Funcionamiento

Para que la exploración cuadrática funcione de manera adecuada y evite los problemas de cobertura incompleta de la tabla, es necesario cumplir ciertas condiciones:

El tamaño de la tabla debe ser un número primo, lo que favorece una mejor distribución de las posiciones generadas por la secuencia de exploración.

Además, el factor de carga debe mantenerse por debajo de 0.5, garantizando que haya suficientes posiciones libres disponibles.

Bajo estas condiciones, se puede asegurar que la secuencia de exploración genera posiciones distintas y que, eventualmente, se encontrará una posición libre para insertar un nuevo elemento.

Análisis de Rendimiento y Conclusión

La exploración cuadrática mejora el rendimiento respecto a la exploración lineal al reducir el efecto del agrupamiento de elementos, lo que permite mantener secuencias de exploración más cortas en promedio.

No obstante, su eficiencia sigue dependiendo del control del factor de carga y de una adecuada elección del tamaño de la tabla.

Análisis de Rendimiento y Conclusión

En términos de complejidad, las operaciones presentan un comportamiento promedio cercano a O(1) cuando el factor de carga es bajo. Sin embargo, en el peor de los casos, el rendimiento puede degradarse a O(n), especialmente si la tabla se encuentra muy cargada o la secuencia no cubre adecuadamente las posiciones.

En conclusión, la exploración cuadrática representa una mejora respecto a la exploración lineal, pero requiere condiciones específicas para garantizar su correcto funcionamiento y eficiencia.

Doble Hashing

El doble hashing es un método de resolución de colisiones en tablas hash con direccionamiento abierto que utiliza dos funciones de dispersión. A diferencia de métodos como la exploración lineal o cuadrática, donde el patrón de búsqueda es fijo, en este caso cada clave genera su propia secuencia de exploración. Esto permite una mejor distribución de los elementos en la tabla y reduce significativamente el problema del agrupamiento.

Idea General del Método

El método se basa en la utilización de dos funciones hash: una función principal h1(x), que determina la posición inicial, y una función secundaria h2(x)h, que define el desplazamiento en caso de colisión. Cuando ocurre una colisión, en lugar de recorrer posiciones consecutivas o aplicar una fórmula cuadrática, el algoritmo utiliza un salto determinado por la clave, lo que genera un recorrido más eficiente.

Función de Exploración

La posición a evaluar en cada intento está dada por la siguiente expresión:

$$H(x,i)=(h1(x)+i \cdot h2(x)) \mod m$$

Esta fórmula unifica el comportamiento del método, ya que contempla tanto la posición inicial (cuando i=0) como las sucesivas posiciones a explorar en caso de colisión.

En la fórmula de doble hashing, h1(x) representa la posición inicial calculada a partir de la clave, mientras que h2(x) determina el tamaño del salto entre intentos. El valor i indica el número de intento, por lo que cada nueva posición se calcula sumando múltiplos del salto definido por la segunda función. Esto genera una secuencia de posiciones distinta para cada clave.

Función de Exploración

Una forma concreta de implementar el doble hashing es:

$$H(x,i)=((x \bmod m)+i \cdot (1+(x \bmod (m-1))))\bmod m$$

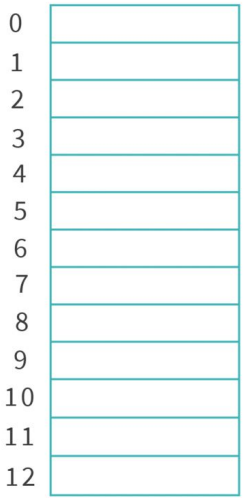
Aquí se observa claramente cómo se combinan ambas funciones. La primera define la posición base y la segunda asegura un desplazamiento distinto de cero. Esta construcción es clave para garantizar que el algoritmo pueda explorar correctamente la tabla.

A partir de la fórmula, se obtiene la siguiente secuencia de exploración:

$$h1(x),\ h1(x)+h2(x),\ h1(x)+2h2(x),\ h1(x)+3h2(x),\ldots$$

Esto muestra que la exploración sigue una progresión aritmética. A diferencia de otros métodos, el incremento no es fijo ni predefinido, sino que depende directamente de la clave, lo que mejora la dispersión de los elementos.

KEYS : 79, 69, 98, 72, 14, 50



Hash table

SCALER Topics

Condiciones de la función h2(x)

Para que el doble hashing funcione correctamente, la segunda función hash debe cumplir:

- No debe devolver 0 - evita quedarse en la misma posición
- Debe generar saltos que recorran distintas posiciones de la tabla
- Debe ser **coprima con el tamaño m** para evitar ciclos

Se recomienda que m sea primo para facilitar estas condiciones.

Una elección práctica es:

$$h2(x)=1+(x \bmod (m-1))$$

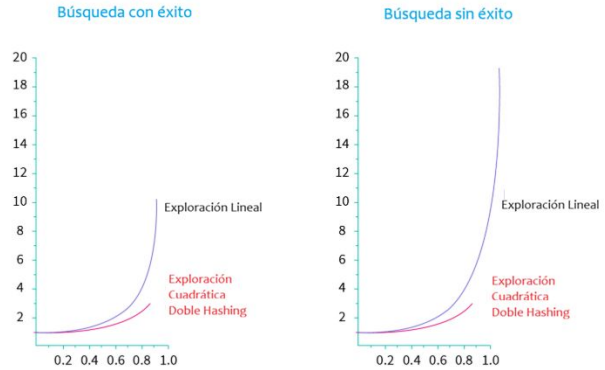
Uso de Otras Funciones Hash

El doble hashing no obliga a utilizar únicamente funciones basadas en el operador módulo. Es posible emplear otros métodos de dispersión, como plegamiento, truncamiento o extracción de dígitos, siempre que se mantenga una buena distribución de las claves.

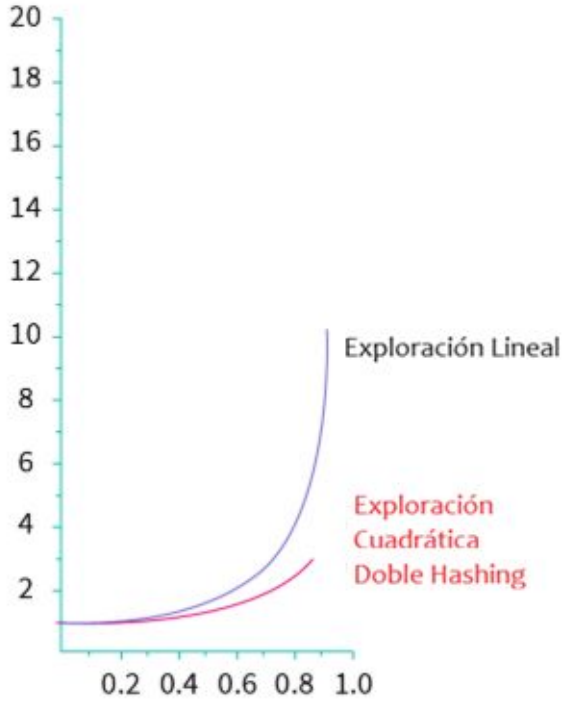
Tanto la función principal como la secundaria deben cumplir ciertas propiedades: deben ser eficientes (tiempo constante), generar valores dentro del rango de la tabla y distribuir uniformemente las claves. Además, es importante que ambas funciones sean lo suficientemente independientes entre sí.

Conclusión

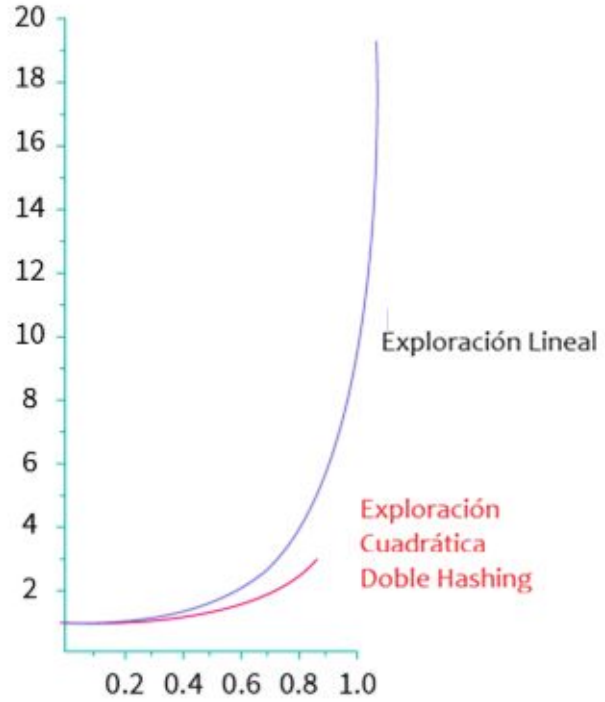
El doble hashing es uno de los métodos más eficientes para resolver colisiones en tablas hash con direccionamiento abierto. Su principal ventaja radica en el uso de una segunda función hash que permite generar secuencias de exploración más dispersas, reduciendo el agrupamiento y mejorando el rendimiento general del sistema.



Búsqueda con éxito

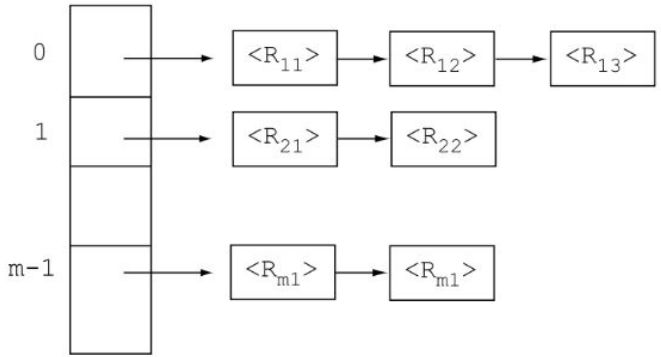


Búsqueda sin éxito



Encadenamiento

En hashing abierto, cada posición de la tabla contiene una estructura que almacena múltiples elementos.
Clásicamente se utiliza una **lista enlazada**, aunque en implementaciones prácticas pueden emplearse otras estructuras como arrays dinámicos o árboles.



Encadenamiento

La estructura de datos empleada para la implementación de una tabla dispersa con direccionamiento enlazado es un array de listas enlazadas. *(podria ser otra estructura en la práctica)*

Cada posición *i* del arreglo almacena una referencia a la dirección de memoria del primer nodo de la lista enlazada correspondiente.

Encadenamiento

En una tabla hash con encadenamiento, las operaciones se realizan en dos etapas: primero se calcula el índice mediante la función hash i=h(x), y luego se opera sobre la lista enlazada asociada a esa posición.

La inserción de un elemento consiste en ubicar la lista correspondiente al índice calculado e incorporar el nuevo nodo. Esta inserción se realiza generalmente al inicio de la lista, evitando recorridos innecesarios y simplificando la operación.

Encadenamiento

Desde el punto de vista de la complejidad algorítmica, la inserción tiene un costo constante O(1), ya que no depende de la cantidad de elementos almacenados, sino únicamente del acceso directo al índice y la inserción en la lista.

No obstante, este análisis supone que no se realizan verificaciones adicionales (como evitar duplicados). En caso de requerirse búsqueda previa, el costo podría incrementarse.

El rendimiento general está fuertemente influenciado por el **factor de carga** $\alpha = n/m$, que representa la cantidad promedio de elementos por posicion del array.

Encadenamiento

La búsqueda de un elemento en una tabla hash con encadenamiento comienza con el cálculo del índice $i=h(x)$, seguido del recorrido de la lista enlazada en esa posición hasta encontrar el elemento o determinar su ausencia.

La complejidad de la búsqueda depende directamente de la longitud de la lista. En promedio, si la función hash distribuye uniformemente los elementos, el costo es $O(1+\alpha)$, donde α es el factor de carga. Esto implica que, para valores pequeños de α, la búsqueda es prácticamente constante.

Sin embargo, en el peor caso, cuando todos los elementos colisionan en la misma posición, la búsqueda puede degradarse a $O(n)$.

Encadenamiento

La eliminación sigue el mismo procedimiento que la búsqueda para localizar el elemento, y luego se elimina ajustando los enlaces de la lista. Su complejidad es equivalente a la búsqueda: $O(1+\alpha)$ en promedio y $O(n)$ en el peor caso.

En conclusión, la eficiencia de las operaciones en hashing abierto depende principalmente de mantener un bajo factor de carga y una buena función hash que minimice las colisiones.

Análisis del Hashing con Encadenamiento

El rendimiento depende directamente del factor de carga λ:

- Búsqueda sin éxito: O(λ)
- Búsqueda con éxito: O(1+λ)
- Inserción: O(1)

Si λ es pequeño, las listas son cortas y las operaciones son cercanas a O(1).
Si λ crece, las listas se alargan y el rendimiento empeora.

Existe un compromiso:

- λ bajo entonces más memoria desperdiciada
- λ alta entonces más colisiones y menor rendimiento

Complejidad y Factor de Carga

En el hashing con encadenamiento, cada posición de la tabla contiene una lista enlazada. Para operar, primero se calcula el índice mediante la función hash i=h(x), y luego se trabaja sobre la lista correspondiente.

La búsqueda consiste en recorrer la lista enlazada. En el peor de los casos, si todos los elementos colisionan en una misma posición, la complejidad es O(n). Sin embargo, con una buena función hash, este caso es poco frecuente.

Para analizar el rendimiento promedio se utiliza el factor de carga $\lambda=n/m$, que representa la cantidad promedio de elementos por posicion del array, no el tamaño real de cada lista.

Rehashing y Optimización

Para mantener un buen rendimiento se utiliza **rehashing**, que consiste en crear una nueva tabla más grande y volver a insertar todos los elementos recalculando su posición.

El objetivo es reducir el factor de carga y mejorar la distribución de los datos, evitando listas largas. Aunque el rehashing tiene un costo de O(n), se realiza de forma ocasional, por lo que su impacto es amortizado.

En resumen, la eficiencia del hashing con encadenamiento depende de:

- una buena función hash
- un factor de carga controlado
- y el uso de rehashing cuando es necesario